GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Sesión 07 — Elección de herramientas

2º CFGS DAW/DAM · Proyecto Intermodular II

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra

🗺️ La sesión en un mapa

Cómo elegir las tecnologías **más adecuadas** para implementar el proyecto: decisiones caras de revertir, tomadas con criterio.

🗄️
SGBD
Relacional vs NoSQL

⚖️
Licencias
MIT · GPL · AGPL · SaaS

📐
Prototipado
LO-FI · HI-FI · MVP

📈
Curvas
Coste de aprender

🧮
Matriz
Decidir con criterios

📝
ADR
Registrar el porqué

ℹ️ Cuatro decisiones con **coste alto de reversión**: si te equivocas en el SGBD o en una licencia, el cambio puede desmontar media aplicación. Y después los aterrizamos en los **casos típicos de PI por ciclo**: DAM, DAW y ASIR.

🧵 TaskFlow: nuestro hilo conductor

📱

App móvil (DAM)
offline-first, sync en segundo plano

🌐

API backend (DAW)
usuarios, sprints, notificaciones y pagos

🛰️

Despliegue (ASIR)
Docker + Ansible, métricas, SSO

ℹ️ El proyecto final del material (tipo Trello/Jira). Cada decisión de esta sesión se comprueba una y otra vez contra él: SGBD, licencias, prototipo y curva.

🧱 ¿Por qué es tan importante elegir bien?

> *Elegir el SGBD o la licencia es como decidir si la casa llevará **electricidad o gas**: si lo decides tarde, no se cambia un enchufe... hay que **abrir las paredes** para pasar tuberías nuevas.*

Si eliges mal...	El coste aparece en...
SGBD	Modelos de datos imposibles, consultas pésimas
Licencia	Te ves obligado a liberar tu producto (o a reescribir)
Prototipo mal definido	3 semanas de diseño que nadie pidió
Curva de aprendizaje	Calendario en el suelo: 3 semanas de estudio no caben en un sprint

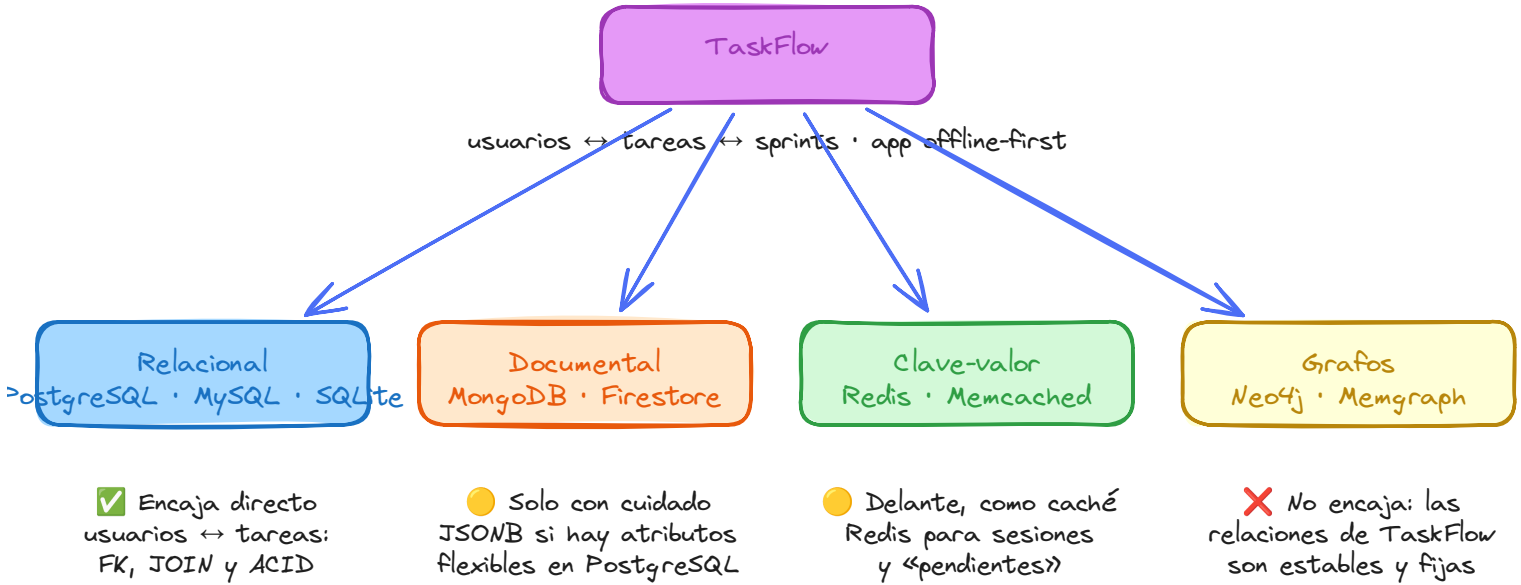
⚠️ A diferencia de un bug (que se arregla en una tarde), las decisiones de herramientas **no se deshacen sin desmontar media aplicación**.

📊 Tipos de SGBD: relacional vs NoSQL

Tipo	Modelo	Fortaleza	Debilidad	Ejemplos
Relacional	Tablas + FK	ACID, `JOIN`s, muy maduro	Esquema rígido	PostgreSQL, MySQL, SQLite
Documental	Documentos JSON	Flexibilidad sin esquema	Sin `JOIN`s, consistencia eventual	MongoDB, Firestore
Clave-valor	Diccionario en memoria	Latencia de microsegundos	Sin relaciones	Redis, Memcached
Grafos	Nodos + aristas	Conexiones profundas	Ecosistema menor	Neo4j

ℹ️ No hay «mejor SGBD»: hay **el adecuado para tu modelo de datos**. Para casi todo lo que haréis en PIM2, la respuesta razonable es relacional.

🗺️ TaskFlow y el ecosistema SGBD



za en PostgreSQL · añade Redis delante si hay tráfico real · NoSQL nativo solo para documentos, grafos o serie

🏭 Taller (relacional) vs almacén (NoSQL)

> ***Analogía 🏭📦:*** *el taller de precisión (relacional) deja cada pieza atornillada en su sitio; el almacén logístico (NoSQL) guarda palés de cualquier forma: mete y saca rapidísimo, pero no le importan las relaciones entre cajas.*

	🏭 Relacional (taller)	📦 NoSQL (almacén)
Esquema	Fijo y estricto: toda pieza en su sitio	Flexible: cada documento es una isla
Relaciones	Atornilladas con FK e integridad	Denormalizadas: duplican datos
Velocidad	Transacciones y consultas complejas	Lecturas/escrituras masivas y simples
Ideal para	Usuarios ↔ tareas, cobros, informes	Catálogos, logs, eventos, sesiones

```
-- La misma pregunta en relacional: cuenta por categoría
SELECT c.nombre, COUNT(p.id)
FROM productos p JOIN categorias c ON c.id = p.categoria_id
GROUP BY c.nombre;
```

ℹ️ En NoSQL, si la categoría está **incrustada** en el producto, cambiarla actualiza **cada documento** uno a uno. Ese es el precio de la flexibilidad.

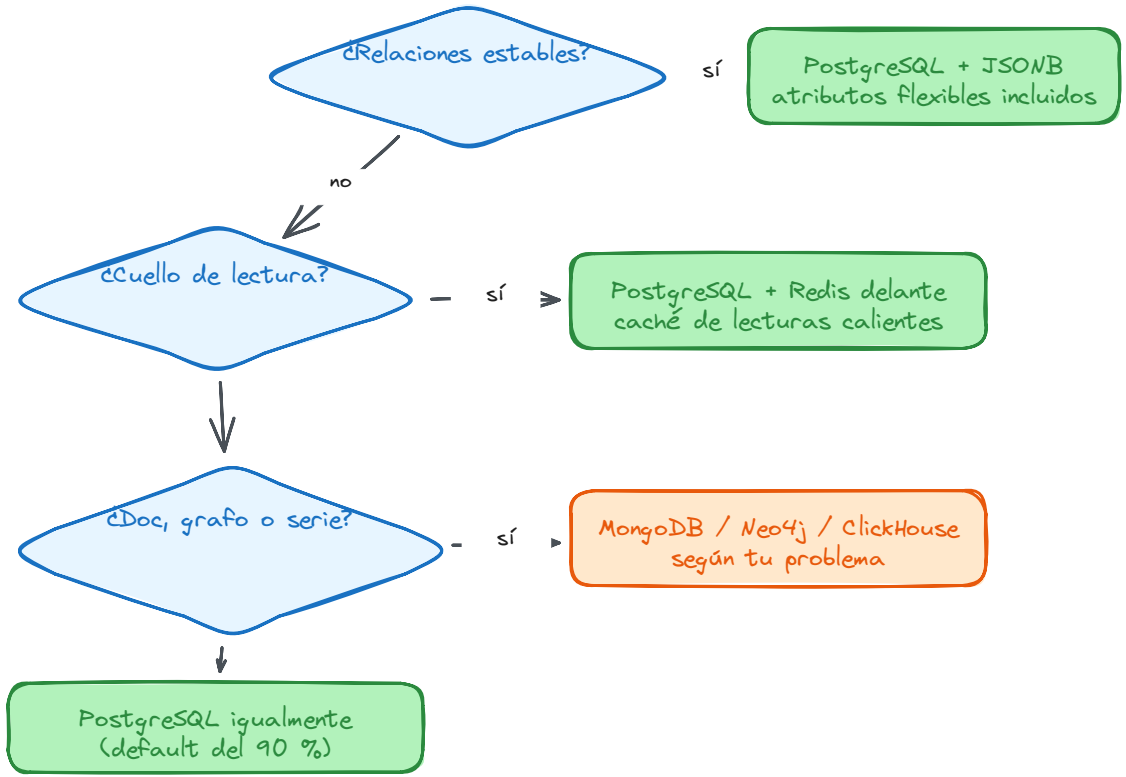
🗄️ Cómo elegir el SGBD de TaskFlow

Regla práctica: el 90 % de los proyectos empieza y acaba bien en PostgreSQL.

1. ¿Entidades con **relaciones estables** (usuarios ↔ proyectos ↔ tareas)? → **Relacional (PostgreSQL)**.
2. ¿Cuello de botella de lectura (1000 req/s en la portada)? → Añade una **capa de caché (Redis)** delante; no cambies de SGBD.
3. ¿Problema **nativo** de grafos, series temporales o documentos sueltos? → Entonces sí, evalúa ese NoSQL concreto.

> ⚠️ Antes de elegir «lo más moderno», preguntad: ¿existe **experiencia en el equipo**, hosting disponible en el aula y comunidad activa? El SGBD más exótico abarata poco y retrasa mucho.

🧭 Árbol de decisión del SGBD

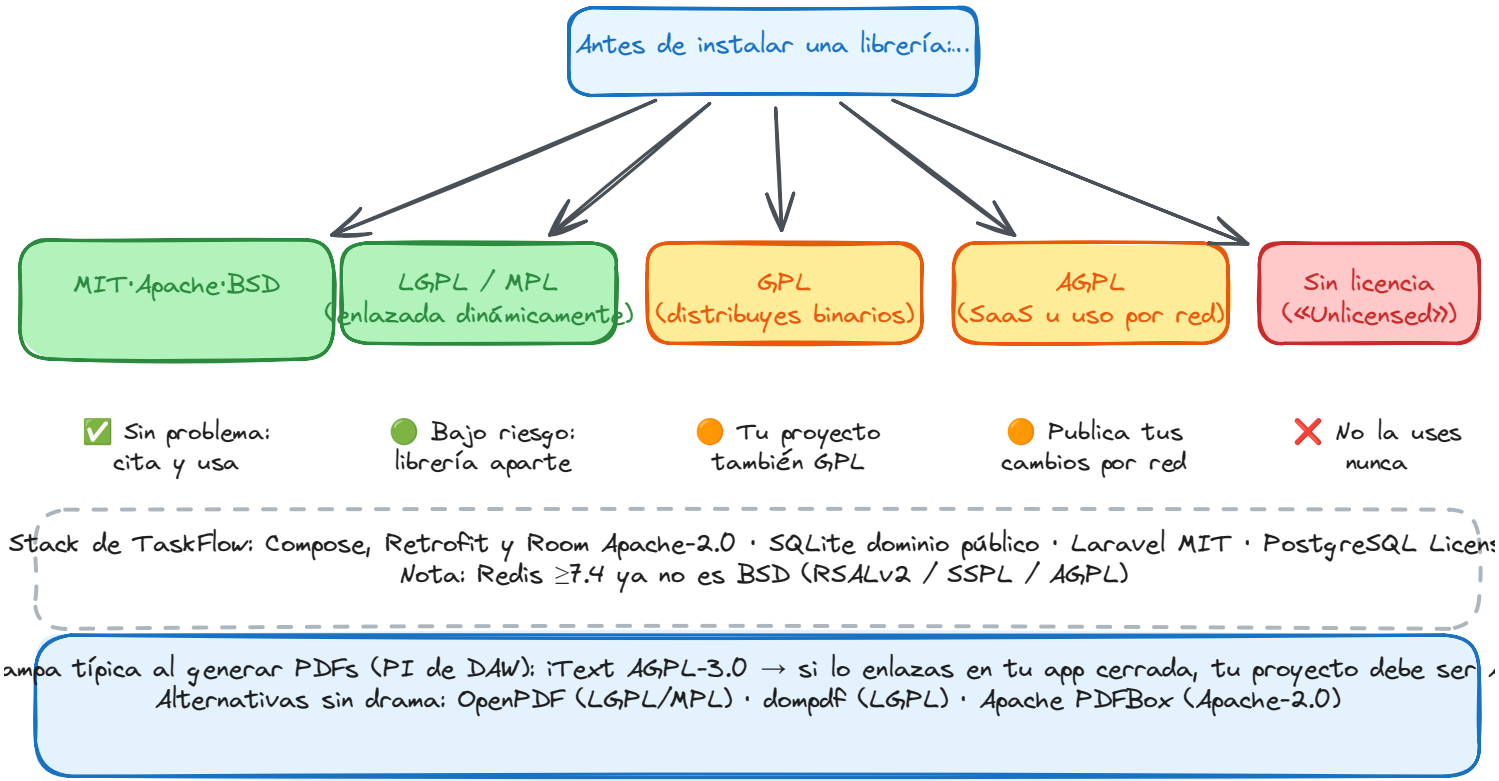


⚖️ Tipos de licencias

Familia	Filosofía	Riesgo	Ejemplos
Permisiva	«Haz lo que quieras, cita al autor»	Mínimo	MIT, Apache-2.0, BSD-3
Copyleft débil (LGPL)	La librería sigue libre; enlazar **dinámicamente** no contamina tu app	Bajo/Medio	LGPL-3.0
Copyleft fuerte (GPL)	Si distribuyes el binario, **tu producto entero** debe ser GPL (viral)	Alto	GPL-3.0
Copyleft de red (AGPL)	GPL + «si se usa **por red**», también aplica: mata el truco SaaS	Muy alto en SaaS	AGPL-3.0
Comercial / SaaS	No compras el código, pagas por el uso	*Vendor lock-in*, costes variables	Licencias de API

ℹ️ MIT/Apache-2.0 dominan el ecosistema JavaScript (React, Vue, Tailwind). La pregunta clave antes de cada dependencia: **¿es compatible con cómo repartiré mi producto?**

📜 Licencias: qué me deja hacer cada una



⚠️ La trampa del copyleft

Imagina que vuestro app Android usa una librería de hashing con licencia **GPL-3.0**:

1. GPL-3.0 exige que **quien recibe el binario (el APK)** reciba también el **código fuente entero** de todo bajo GPL... incluida vuestra app.
2. Si tu modelo de negocio no quiere publicar el código, **tienes un problema**: la licencia contamina el producto final.
3. Soluciones: cambia a una librería **MIT/Apache**, o encapsula la GPL en un **servicio separado** (comunicación por HTTP), si la licencia lo permite.

> ⚠️ Y el caso peor: sin licencia (`Unlicensed`) el código NO es libre — todos los derechos reservados. **No lo uses. Nunca.**

📐 Prototipado: definir antes de construir

Un prototipo es una versión **rápida y barata** de la idea, hecha para **aprender** antes de invertir.

Nivel	Qué es	Herramienta	¿Cuándo parar?
LO-FI (wireframe)	Bocetos de baja fidelidad	Papel, Excalidraw, Balsamiq	Cuando el **flujo** está claro
HI-FI (interactivo)	Maqueta clicable casi final	Figma	Cuando el **diseño** convence
PoC (prueba de concepto)	Código mínimo que demuestra viabilidad	Proyecto aparte	Cuando se supera el «¿podemos?»
MVP	Producto real y desplegado con riesgos integrados	El stack real	Cuando hay **usuarios reales**

ℹ️ El arquitecto no te enseña el edificio a precio de hormigón: te enseña la **maqueta de cartón**. El prototipo es la maqueta del software.

♻️ Prototipo desechable vs. evolutivo

🗑️ Desechable (throwaway)

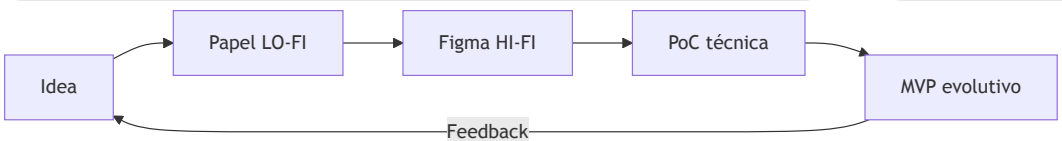
Su misión es **aprender y tirarse**. El código es «sucio» a propósito: eso lo hace rápido.

Regla para PIM2: los prototipos de Figma y las PoC son **desechables**.

🔄 Evolutivo

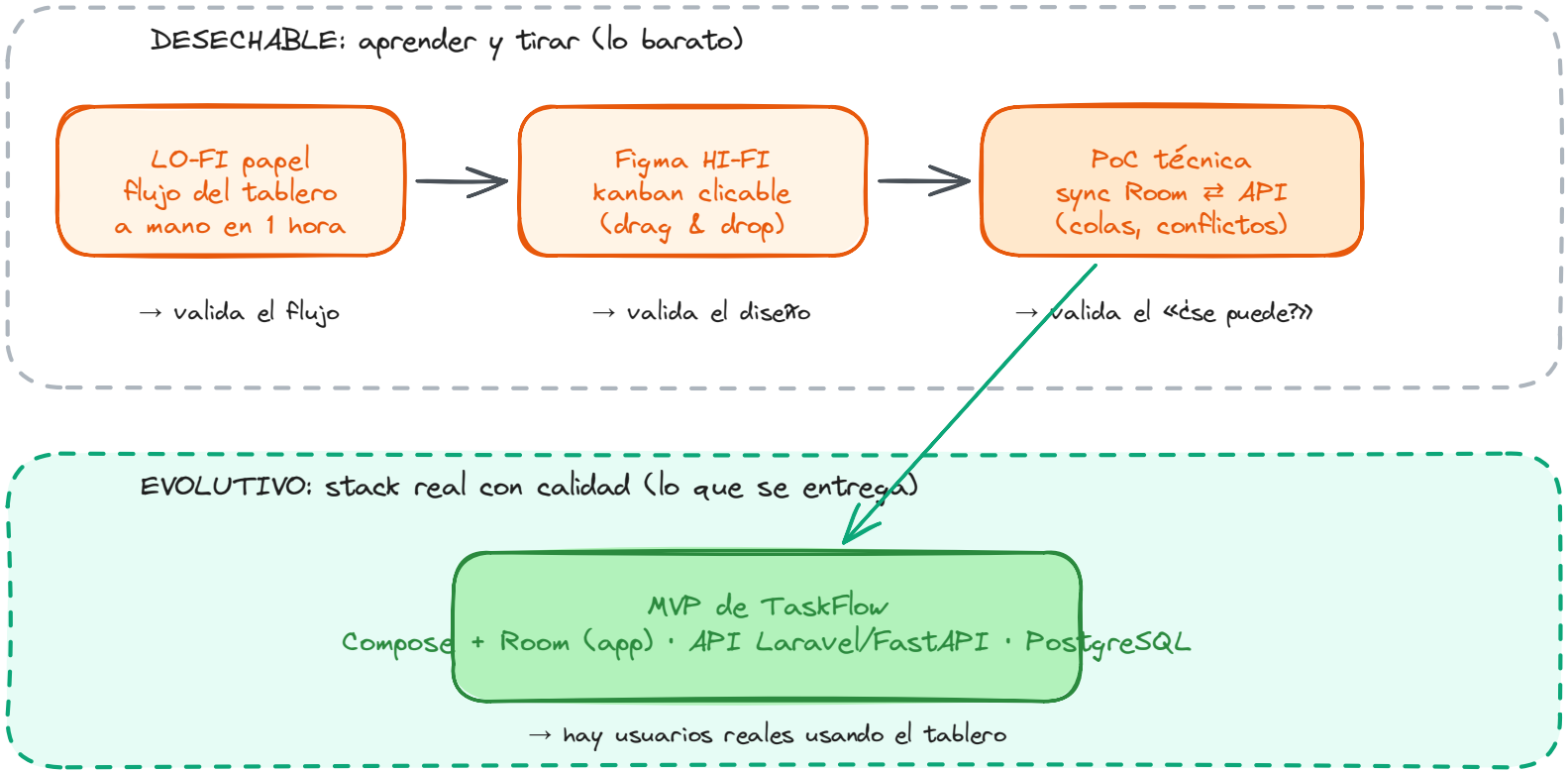
Se construye con el **stack real** y con calidad, porque va a **crecer hasta ser el producto**.

Regla para PIM2: vuestro MVP es la versión evolutiva del prototipo… ahora con tests y arquitectura.



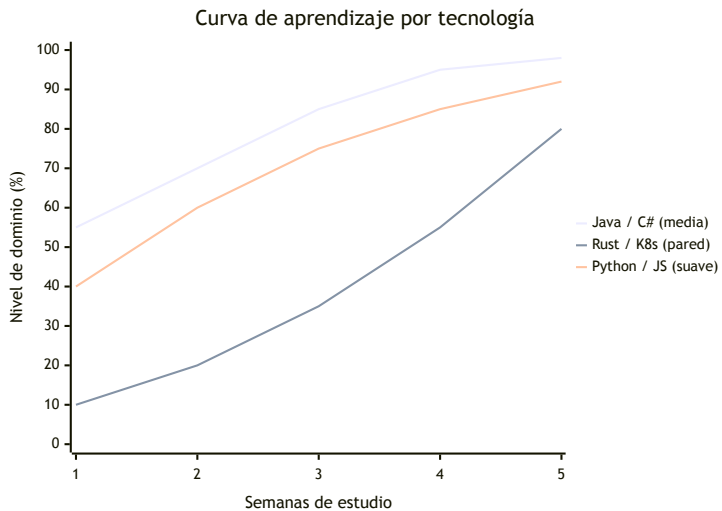
⚠️ Error clásico: disfrazar el prototipo de producto final y entregarlo con deuda incrustada. Si es un prototipo de papel y Figma, **no lo reutilices como código**.

De LO-FI a MVP: TaskFlow, fase a fase



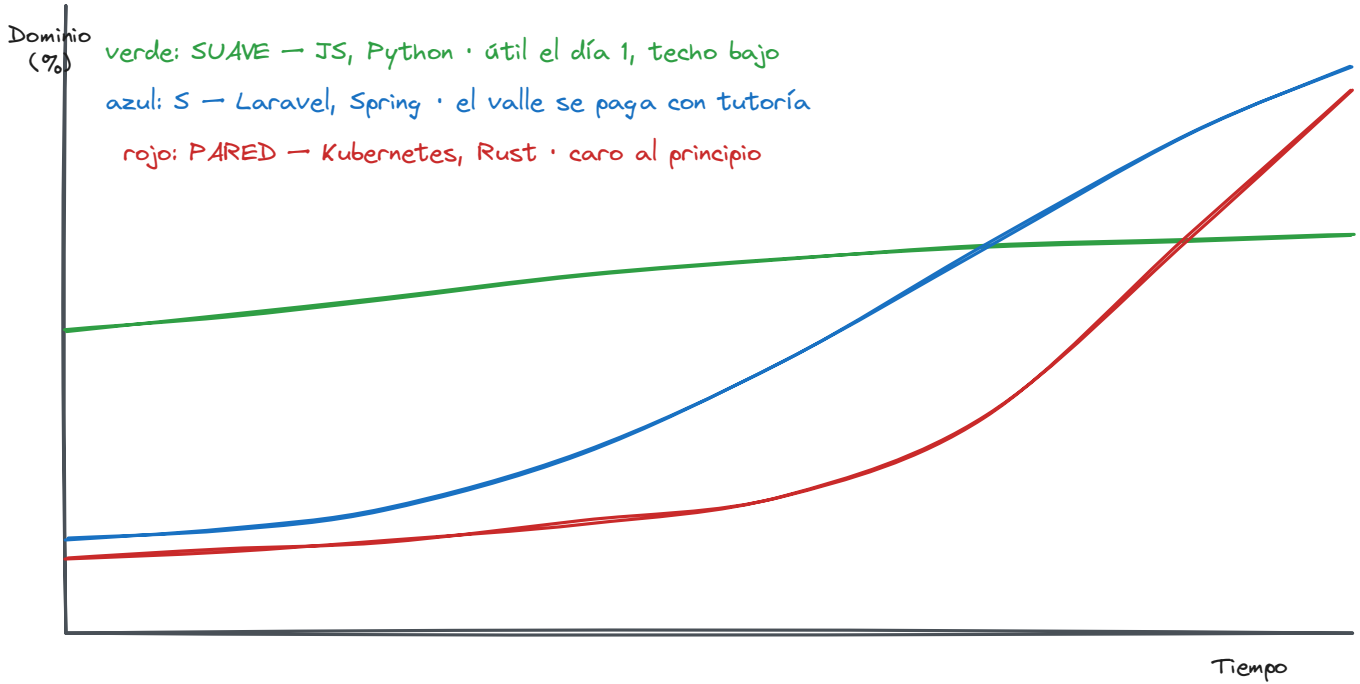
📈 Curvas de aprendizaje

La curva modela cómo crece el **dominio** (eje Y) con el **tiempo invertido** (eje X). Su forma **importa** cuando hay un calendario fijo.



ℹ️ La curva «suave» te hace **productivo el primer día**; la «de pared» (Rust, Kubernetes) es frustrante al principio pero alcanza un techo mucho más alto a largo plazo.

📉 Las tres curvas, en un vistazo



En PIM2: la curva no la paga el proyecto, la paga tu calendario — elige curvas que el equipo ya domine

🚲 Camión vs. bicicleta: cómo influye en la elección

🚛 Framework completo (Laravel, Spring)

Como aprender a llevar un **camión**: empiezas perdido (ORM, rutas, migraciones) pero transportas lo que sea.

Cuando encaja: proyectos de 4-6 meses con **tutoría** y comunidad.

🚲 Framework micro (Express, Flask)

Como una **bicicleta**: en 10 minutos ya te mueves.

El problema: cuando el proyecto crece, tendrás que improvisar el «camión» tú mismo... **y mal**.

Curva	Cuándo elegirla	Cuándo evitarla
Suave	Plazos cortos, prototipos, equipo junior	Sistemas de altísima concurrencia
Pared	Proyectos de larga vida, requisitos extremos	Plazos de 3 meses (¡como PIM2!)
S (framework)	4-6 meses con tutoría	«Tengo 2 semanas» (valle = muerte)

ℹ️ La curva de aprendizaje **no la paga el proyecto: la paga vuestro calendario**. Elegir lo que el equipo ya domina —aunque no sea lo más moderno— es una decisión redonda.

📱 Casos de PI: DAM (app nativa)

App Android que gestiona algo concreto, a menudo offline-first.

Caso	SGBD	Stack típico	Curva
Inventario de pequeño comercio	SQLite embebida	Kotlin + Compose + Room	Suave
App de citas/turnos (clínica, taller)	PostgreSQL + SQLite local	Compose + REST (Laravel/FastAPI)	Media
Gestor de la FCT (horas, tareas)	PostgreSQL	Compose + REST + JWT	Media

ℹ️ Licencias: Android y Compose son Apache/MIT sin drama; lo que vigila son los SDK **propietarios** (Firebase, Play Billing, mapas o pagos). **Prototipo**: Figma de pantallas (desechable) → MVP evolutivo con Compose + Room. **¿Y TaskFlow?** Es el caso «citas/turnos»: app offline-first que sincroniza contra su API.

🌐 Casos de PI: DAW (plataforma web)

Portal web con roles y panel administrativo: PostgreSQL casi siempre.

Caso	SGBD	Stack típico	Curva
Reservas online (pistas, citas)	PostgreSQL + Redis (caché)	Laravel o Express + Vue/React	Media
Gestión de club deportivo (socios, cuotas)	PostgreSQL	Laravel + Blade/Vue	Media
Tienda / catálogo con panel admin	PostgreSQL (o MongoDB si muta a diario)	Vue/React + Express/Laravel	Suave/Media
Portal de incidencias (tickets)	PostgreSQL	Laravel/Vue + notificaciones	Media

⚠️ Licencias: el stack web es MIT/Apache sin drama; el riesgo entra por **librerías auxiliares** (PDFs: iText AGPL, OpenPDF/dompdf LGPL) o por **plantillas comerciales** (una licencia = un proyecto final). **Prototipo**: Figma desechable + PoC de pagos aparte; MVP evolutivo con el stack real.

🛰️ Casos de PI: ASIR (servicios en red)

Desplegar y documentar un servicio completo. El SGBD lo elige el servicio, no tú.

Caso	SGBD del servicio	Stack típico	Curva
Nextcloud + Office online	PostgreSQL (oficial)	Docker + Nginx + Ansible	Media
Monitorización de red	Prometheus + Grafana (SQLite interno)	Prometheus + Grafana + alertas	Media
SSO corporativo (Keycloak + LDAP)	PostgreSQL	Keycloak + Ansible	Media
VPN + firewall (pfSense/WireGuard)	No aplica (SQLite interno)	pfSense + reglas + backups	Suave

⚠️ Licencias por **servicio**: Nextcloud y Grafana son **AGPL** (desplegar sin tocar su código → sin problema; modificar y ofrecer por red → publicar cambios). Keycloak Apache-2.0, pfSense BSD, Ansible GPL (tus playbooks YAML son datos). **Prototipo**: lab con snapshot (desechable) → infra con IaC documentada (evolutiva). **Kubernetes es «pared»: evitarlo en PI.**

🛠️ Herramientas de desarrollo: el entorno

Área	Opciones típicas	Licencia / coste	En PIM2
Editor / IDE	VS Code · IntelliJ/Android Studio	VS Code: MIT (núcleo) · AS/IntelliJ CE: Apache-2.0	Android Studio en DAM; VS Code en web
Control de versiones	Git + GitHub/GitLab	Git GPL · hosting SaaS (plan gratuito, pack de estududiante)	obligatorio: sin repo, no hay PI
Terminal / entorno	Linux · WSL2	libres	para ASIR es parte del servicio
Gestión del proyecto	Trello · GitHub Projects	freemium	puede **ser el propio producto** (TaskFlow)

ℹ️ Se decide **ya** y se registra en otro ADR: los atajos, plugins y la IA se ajustan a ti. Y ojo: **gratis ≠ libre** — el Marketplace y el hosting te traen condiciones propias.

🤖 Agentes de IA: quién es quién

Agente	Qué es	Licencia / coste	Encaja si…
opencode	Agente CLI/TUI open source	OSS (MIT) · modelo *BYOK*	privacidad, hasta modelos locales
Codex (OpenAI)	Agente en ChatGPT + CLI	herramienta libre · servicio por suscripción	ya usas ChatGPT
Claude Code (Anthropic)	Agente en terminal/IDE, MCP	propietaria · Claude Pro/Max o API	flujo agéntico potente
Antigravity (Google)	IDE agéntico (sobre VS Code)	gratuito (preview), Gemini integrado	probar el flujo completo sin pagar
Freebuff	Agente gratuito en el chat del proyecto	gratuito, con modelo	empezar sin instalar nada

⚠️ Elegir agente = decisión de herramienta: **privacidad** (¿sale tu código del equipo?), **cuota** y **trazabilidad** (commits y PR revisables; nada de secretos en los prompts). La IA ayuda a escribir el código; **el ADR sigue siendo tuyo**.

🧮 La matriz de decisión

Ejemplo real: el SGBD de **TaskFlow**. Convertir la intuición en **criterios explícitos con pesos**, y documentar el resultado en la memoria del proyecto. Funciona igual con un PI de DAM (la app), de DAW (la web) o de ASIR (el despliegue): lo que cambia es la lista de candidatos y los pesos.

CRITERIO (peso)	PostgreSQL	MongoDB	SQLite
Encaje con el modelo (x4)	5	3	4
Coste de aprendizaje (x3)	4	3	5
Ecosistema/comunidad (x2)	5	4	3
Licencia compatible (x3)	4	4	5
Herramientas del aula (x1)	5	3	5
TOTAL PONDERADO	47	31	45

1. Lista los candidatos (mínimo 2-3 alternativas razonables).
2. Puntúa del 1-5 cada criterio y **pondera** según la importancia para el proyecto.
3. Documenta el porqué como **ADR** (mini-ficha de decisión) en la memoria: contexto → decisión → consecuencias.

✅ Resumen de la sesión

- El SGBD se elige por el **modelo de datos**: relacional para TaskFlow, caché (Redis) si hay cuello de botella, NoSQL solo si tu problema es nativo documental/gráfico/columnar.
- Las licencias: **MIT/Apache sin drama**; GPL/AGPL «contaminan» el producto si lo distribuyes; **sin licencia = inutilizable**.
- El prototipo valida la idea barato y rápido (**LO-FI/HI-FI desechable**); el MVP con el stack real es **evolutivo**.
- Las curvas de aprendizaje condicionan el calendario: no eliges la tecnología más potente, eliges **la que el equipo domina a tiempo**.
- El **entorno** y los **agentes de IA** son herramientas más: licencia, cuota y **privacidad del código** (gratis ≠ libre).
- Toda elección se pasa por la **matriz de decisión** y se registra en la memoria como **ADR**.

ℹ️ El coste de elegir mal no es el tiempo perdido: es que las decisiones de herramientas **no se deshacen sin desmontar media aplicación.**

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //